KiteLang - Compiler Construction Project

Detailed Linux build, usage, testing, and language-reference guide for the Phase 1 interpreter and Phase 2 LLVM AOT compiler.

1. Purpose and project phases

KiteLang is a Basant-inspired language for algorithmic programming. It combines C-style functions, variables, arrays, expressions, and braces with pecha (if), warna (else), dheel (while), dor (inclusive range loop), and palta (swap).

Phase 1 uses Flex and Bison to parse a program and execute its AST directly. Phase 2 lowers supported AST nodes into textual LLVM IR, which Clang compiles to a native executable.

2. Linux requirements

Install build-essential, Flex, Bison, Clang, and LLVM. A C++17 compiler is required.

```
sudo apt update
sudo apt install -y build-essential flex bison clang llvm
flex --version
bison --version
clang --version
```

3. Build

```
make
# Optional: make interpreter
# Optional: make compiler
# Cleanup:  make clean
```

The build creates build/kitelang-interpreter for Phase 1 and build/kitelang-compiler for Phase 2.

4. Phase 1 - interpreter

```
./build/kitelang-interpreter examples/insertion_sort.kite
```

Expected output is one number per line: 1, 2, 4, 7, 8, 9. No intermediate code is generated in this phase.

5. Phase 2 - AOT compiler

```
./build/kitelang-compiler examples/insertion_sort.kite -o build/insertion.ll
clang build/insertion.ll -o build/insertion
./build/insertion
```

The compiler emits LLVM IR. Clang creates a native executable. If -o is omitted, the compiler writes out.ll.

6. Language reference

```
int values[3] = {9, 4, 1};
pecha (values[0] > values[1]) {
    palta(values[0], values[1]);
} warna {
    print(values[0]);
}
dheel (values[0] > 0) { values[0] = values[0] - 1; }
dor (i : 0 -> 2) { print(values[i]); }
```

Functions use C-style parameters and braces; array parameters use []. Supported declarations are int, float, bool, and void. The implemented runtime uses integer representation, so float and bool keywords currently have integer semantics.

Arithmetic operators are +, -, *, /, and %. Comparison operators are less-than, greater-than, less-or-equal, greater-or-equal, equality, and inequality. Zero is false; any nonzero integer is true. print(expression) writes one integer and a newline.

7. Arrays, loops, and swap

Arrays are fixed-size, one-dimensional, and zero-indexed. Keep indexes in bounds. dor evaluates its inclusive start and end bounds before its loop, then increases its loop variable by one. palta(left, right) exchanges variables or array elements and lowers to ordinary loads and stores in LLVM IR.

8. Test procedure

```
make test
```

The test builds both phases, verifies interpreter output against tests/expected_insertion_sort.txt, emits build/insertion.ll, builds a native executable with Clang, runs it, and checks its output against the same expected file. A zero exit status means the end-to-end demonstration passed.

9. Project files

src/kitelang.l: Flex lexer. src/kitelang.y: Bison grammar, AST interpreter, and LLVM emitter. src/ast.hpp: shared AST declarations. examples/insertion_sort.kite: primary demonstration. examples/array_smoke.kite: smoke test. tests/expected_insertion_sort.txt: expected test output. Makefile: build commands. KiteLang_Proposal.pdf: Phase 0 proposal.

10. Troubleshooting and scope

Install missing Flex, Bison, or Clang packages using the requirements command. KiteLang syntax errors include a source line number; check semicolons, braces, and keyword spelling. The project supports integer algorithm programs with fixed one-dimensional arrays in one source file. Pointers, dynamic memory, multidimensional arrays, external libraries, classes, and multi-file programs are outside scope.